

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

По учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции
программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Миниханов Вадим Павлович
(подпись) (фамилия, имя, отчество)

Группы ИПс-21

Руководитель практической подготовки от профильной организации

Преподаватель _____
(должность)

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

МП

«26» июня 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ.....	5
1.1. Сбор информации о существующем состоянии продукта	5
1.2. Разработка технического задания	9
РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ	14
РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ.....	23
РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	25
РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ.....	28
ЗАКЛЮЧЕНИЕ.....	31
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	33
ПРИЛОЖЕНИЯ	36

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем

В настоящей работе представлен отчет о прохождении учебной практики студента-практиканта. Основными видами деятельности являлись: Разработка программного обеспечения; выполнение отладки программного модуля с использованием специализированных программных средств; осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; уметь производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент;
2. Выполнять интеграцию модулей в программное обеспечение;
3. Выполнять отладку программного модуля с использованием специализированных программных средств;

4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.
5. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования
6. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
7. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
8. Планировать и реализовывать собственное профессиональное и личностное развитие;
9. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами
10. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
11. Использовать информационные технологии в профессиональной деятельности
12. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ

1.1. Сбор информации о существующем состоянии продукта

В рамках прохождения производственной практики мне было выдано задание на разработку настольного приложения для управления задачами. Данное приложение должно помочь пользователям более эффективно организовывать свою повседневную деятельность и контролировать выполнение поставленных задач. В ходе анализа требований я определил следующий перечень функций, которые необходимо реализовать в приложении:

1. Возможность создания задач с указанием названия, описания, приоритета, срока выполнения и текущего статуса.
2. Отображение всех задач в виде удобного списка для их просмотра и отслеживания.
3. Фильтрация задач по статусу выполнения: «Новая», «В процессе», «Завершена».
4. Реализация полнотекстового поиска по названию и описанию задачи.
5. Редактирование и удаление уже существующих задач.
6. Сохранение и загрузка данных в формате JSON-файла.
7. Вынесение основной бизнес-логики в отдельную подключаемую библиотеку.
8. Написание модульных тестов для проверки корректности работы приложения.

Далее было дано описание бизнес-процесса «AS IS»

Прежде чем приступить к разработке, я провёл небольшое исследование того, какими способами люди в настоящее время ведут учёт своих задач и дел. Оказалось, что большинство пользователей прибегают к одному из трёх распространённых подходов, каждый из которых имеет свои существенные недостатки.

Способ 1: Бумажные записи

Наиболее традиционный способ — записывать задачи в блокноты или на бумажные стикеры, которые крепятся на видное место. Несмотря на свою простоту, данный метод крайне ненадёжен: записи легко теряются, рвутся или просто остаются незамеченными. Кроме того, при большом количестве задач найти нужную запись становится крайне затруднительно, а возможность сортировки по степени важности и вовсе отсутствует.

Способ 2: Электронные таблицы (Excel)

Более продвинутые пользователи ведут учёт задач в электронных таблицах. Это несколько лучше бумажного варианта, однако и здесь есть серьёзные неудобства. Данные приходится обновлять вручную, что занимает немало времени. Поиск по большой таблице неудобен, а наглядного визуального выделения важных или просроченных задач, как правило, нет.

Способ 3: Устные договорённости

В некоторых случаях задачи вообще не фиксируются — они передаются устно. Это самый ненадёжный вариант: задачи забываются, детали искажаются, а проконтролировать их выполнение практически невозможно.

Ключевые проблемы текущего подхода:

- Задачи хранятся в разрозненных местах — блокноты, таблицы, стикеры, переписки
- При большом количестве задач найти нужную крайне сложно
- Отсутствует удобная фильтрация по статусу выполнения
- Невозможно быстро понять, какие задачи уже просрочены
- Нет механизма выделения приоритетных или важных задач

- Отсутствует общая статистика по задачам — картина в целом непрозрачна

Таблица 1 – Описание бизнес-процессов

№	Операция	AS IS (текущая)	TO BE (целевая)	Улучшение
1	Появление задачи	Фиксация в сознании	Ввод в систему	Автоматизация
2	Запись задачи	Ручная (бумага/Excel)	Заполнение формы	Структуризация
3	Хранение	Разрозненное	JSON-файл	Централизация
4	Поиск	Ручной перебор	Поисковая строка	Эффективность
5	Фильтрация	Отсутствует	По статусу	Удобство
6	Сортировка	Отсутствует	Автоматическая	Порядок
7	Контроль сроков	Ручной (человек)	Автоматический	Надежность
8	Статистика	Отсутствует	Автоматическая	Аналитика
9	Редактирование	Ручное переписывание	Через интерфейс	Скорость
10	Удаление	Потеря/забывание	С подтверждением	Безопасность

Далее рассмотрим описание процесса «ТО BE» (Как должно быть после внедрения).

Проанализировав все вышеперечисленные проблемы, я пришёл к выводу, что разрабатываемое приложение должно устранить их в полной мере. Ниже я описываю, как именно будет работать моя программа.

Пользователь открывает специальное диалоговое окно, в котором заполняет все необходимые поля: название, описание, приоритет, срок выполнения и статус. После нажатия кнопки «Сохранить» задача автоматически добавляется в список и сохраняется.

Все добавленные задачи отображаются в виде наглядного списка. Для каждой задачи показывается её название, описание, статус, приоритет и срок выполнения. Важные задачи выделяются значком звёздочки, а просроченные — визуальной подсветкой.

В приложении реализована строка поиска, позволяющая находить задачи по названию или описанию. Помимо этого, предусмотрена фильтрация по статусу и сортировка — по приоритету или сроку выполнения.

Пользователь может в любой момент открыть задачу и изменить любые её поля. Удаление реализовано с запросом подтверждения, что исключает случайную потерю данных.

В нижней части главного окна постоянно отображается актуальная статистика: общее количество задач, количество новых, выполняемых, завершённых, просроченных и важных.

Все задачи сохраняются в JSON-файл, что обеспечивает их надёжное хранение. Предусмотрена возможность как сохранения, так и загрузки данных из файла.

Таблица 2 – Описание бизнес-процессов

Что сравниваем	Как было (AS IS)	Как стало (TO BE)	Что улучшилось
Хранение задач	В разных местах	В едином JSON-файле	Всё в одном месте
Поиск задач	Долго и неудобно	Строка мгновенного поиска	Быстро и легко
Фильтрация	Отсутствует	По статусу выполнения	Удобный отбор задач
Сортировка	Отсутствует	По приоритету и сроку	Логичный порядок
Контроль сроков	Отсутствует	Визуальная подсветка	Просрочка заметна сразу
Важные задачи	Не выделяются	Отмечаются звёздочкой	Приоритеты представлены
Статистика	Отсутствует	Счётчики в реальном времени	Наглядно и информативно
Надёжность данных	Высокий риск потери	Сохранение в файл	Данные защищены
Удобство работы	Сложно и неудобно	Интуитивный интерфейс	Легко освоить

1.2. Разработка технического задания

Техническое задание — это основополагающий документ, в котором формально описываются все требования к разрабатываемому программному обеспечению. По сути, ТЗ является своеобразным договором между заказчиком и разработчиком, в котором чётко прописано, что именно должна уметь делать программа, каким техническим требованиям она должна соответствовать и как будет проверяться её работоспособность. Без технического задания разработка ведётся «вслепую», что неизбежно приводит к ошибкам и переделкам.

Таблица 3 – ГОСТ 19.XXX (ЕСПД) — единая система программной документации

ГОСТ	Что описывает
ГОСТ 19.201-78	Техническое задание: содержание и оформление
ГОСТ 19.101-77	Виды программ и программных документов
ГОСТ 19.102-77	Стадии разработки программ

Таблица 4 – ГОСТ 34.XXX — стандарты для автоматизированных систем

ГОСТ	Что описывает
ГОСТ 34.602-89	Техническое задание на создание автоматизированной системы
ГОСТ 34.603-92	Виды испытаний автоматизированных систем

Содержание технического задания

1. Общие положения

Наименование программного продукта: «Планировщик задач».

Заказчик: руководитель производственной практики.

Исполнитель: обучающийся (студент).

Сроки реализации: разработка осуществляется в период, установленный графиком прохождения производственной практики.

2. Основания для проведения разработки

Основанием для разработки является индивидуальное задание, утверждённое руководителем практики в рамках производственной

практики. В соответствии с заданием предусмотрена самостоятельная реализация настольного приложения, ориентированного на управление задачами, с применением технологического стека C# и WPF.

3. Целевое назначение программы

Программный продукт предназначен для автоматизации процессов планирования и контроля личных и служебных задач. Функционал приложения обеспечивает выполнение следующих операций: добавление, редактирование и удаление задач, мониторинг временных сроков их исполнения, а также оперативный поиск необходимых записей с использованием механизмов фильтрации и контекстного поиска.

4.1. Функциональные требования

В таблице 5 представлен перечень функциональных возможностей программного продукта с описанием способов их реализации.

Таблица 5 – Функциональные требования к приложению

№	Функция	Описание реализации
1	Добавление задачи	Пользователь заполняет форму, содержащую следующие поля: наименование, описание, приоритет, срок исполнения, статус.
2	Просмотр задач	Все задачи выводятся в виде списка с отображением ключевых атрибутов.
3	Фильтрация	Выбор статуса из выпадающего списка обеспечивает отображение только тех задач, которые соответствуют выбранному значению.
4	Поиск	Ввод текста в поисковую строку инициирует фильтрацию задач по вхождению подстроки в наименование или описание.
5	Редактирование	При выборе задачи предоставляется возможность изменения любого из её полей.
6	Удаление	Удаление задачи осуществляется после подтверждения операции в диалоговом окне.
7	Сортировка	Предусмотрена сортировка задач по приоритету или по сроку выполнения.

№	Функция	Описание реализации
8	Отметка важных задач	Задача может быть помечена как важная, что визуально отображается с помощью значка-звёздочки.
9	Статистика	В нижней панели интерфейса автоматически отображается количественное распределение задач по статусам.
10	Сохранение	Реализован экспорт всех задач в JSON-файл через диалоговое окно выбора пути.
11	Загрузка	Обеспечен импорт задач из JSON-файла с автоматическим обновлением списка.

4.2. Технические требования

К программному продукту предъявляются следующие технические требования:

- язык реализации – C#;
- целевая платформа – .NET с использованием Windows Presentation Foundation (WPF);
- архитектурное решение – бизнес-логика вынесена в отдельную библиотеку классов Core.dll;
- формат хранения данных – JSON-файл.

4.3. Требования к пользовательскому интерфейсу

Пользовательский интерфейс должен отвечать принципам минимализма и интуитивной понятности. Все управляющие элементы снабжаются однозначными подписями, исключающими двусмысленное толкование. Визуальное оформление выполняется в сдержанном стиле, без избыточных декоративных элементов. Просроченные задачи должны визуально выделяться в общем списке для оперативного привлечения внимания пользователя.

4.4. Требования к надёжности

В отношении надёжности программного продукта установлены следующие требования:

- недопустимость аварийного завершения работы при возникновении исключительных ситуаций;

- при ошибках чтения или записи файлов пользователю должно выводиться информативное сообщение с описанием характера ошибки;
- данные должны корректно сохраняться и восстанавливаться из файла без потери целостности;
- все поля ввода подлежат базовой валидации перед сохранением с целью предотвращения некорректных значений.

6. Состав и содержание работ

Процесс разработки включает следующие этапы, представленные в таблице 6.

Таблица 6 – Этапы выполнения работ

Этап	Содержание работ
1	Проектирование архитектуры программного продукта и структуры проекта
2	Разработка библиотеки бизнес-логики (Core.dll)
3	Разработка пользовательского интерфейса на основе WPF
4	Написание модульных тестов для проверки ключевых компонентов
5	Подготовка технической документации и отчётных материалов

7. Порядок контроля и приёмки

Контроль работоспособности программного продукта осуществляется посредством ручного тестирования по следующим сценариям:

- запуск приложения и проверка корректности отображения интерфейса;
- добавление новой задачи с проверкой её появления в общем списке;
- выполнение поиска по наименованию и описанию задачи;
- фильтрация списка по каждому из доступных статусов;
- редактирование существующей задачи с сохранением изменений;
- удаление задачи с обязательным подтверждением действия;

- сохранение задач в JSON-файл и последующая загрузка с проверкой восстановления данных.

В рамках данного раздела был проведён анализ существующих подходов к организации управления задачами, выявлены их основные недостатки, а также определены способы их устранения на уровне проектных решений разрабатываемого приложения. На основе проведённого анализа составлено техническое задание, фиксирующее совокупность функциональных, технических и эксплуатационных требований к программному продукту.

По итогам проведённого анализа может быть сделан следующий вывод: разрабатываемое приложение «Планировщик задач» обеспечивает комплексное решение ключевых проблем, характерных для существующих аналогов. Централизованное хранение всех задач в едином JSON-файле гарантирует целостность и сохранность данных. Встроенный механизм поиска позволяет оперативно находить необходимые записи, а инструменты фильтрации и сортировки обеспечивают гибкое управление списком в зависимости от текущих потребностей пользователя.

Визуальная дифференциация просроченных задач способствует своевременному контролю соблюдения сроков. Возможность маркировки важных задач с помощью специального индикатора (значка-звёздочки) обеспечивает их приоритизацию в общем списке. Автоматически обновляемая статистическая панель предоставляет пользователю актуальную информацию о распределении задач по статусам, формируя целостное представление о текущей загрузке. Механизмы сохранения и загрузки данных гарантируют надёжное восстановление состояния приложения, что подтверждает практическую применимость разработанного решения.

РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

В ходе реализации приложения «Планировщик задач» в качестве методологической основы была выбрана итерационная модель разработки программного обеспечения. Данная модель предполагает декомпозицию всего процесса разработки на последовательность непродолжительных повторяющихся циклов (итераций), по завершении каждой из которых формируется работоспособная, хотя и функционально неполная версия продукта.

Выбор итерационной модели обусловлен следующими факторами. Во-первых, требования к приложению могли уточняться и корректироваться в процессе разработки, что делало применение жёсткой каскадной модели нецелесообразным. Во-вторых, итерационный подход обеспечивал возможность последовательного наращивания функциональности: на начальном этапе была реализована базовая структура проекта и модели данных, затем разработана бизнес-логика в составе библиотеки Core, после чего создан пользовательский интерфейс и, наконец, проведено тестирование. В-третьих, итерационная модель допускала возврат к предыдущим этапам при обнаружении ошибок или необходимости доработок без нарушения целостности общей структуры проекта.

В таблице 7 представлены основные итерации разработки с указанием содержания работ и полученных результатов.

Таблица 7 – Этапы реализации приложения

Итерация	Содержание работ	Результат
1	Проектирование архитектуры, создание структуры решения	Сформирована структура решения, включающая три проекта
2	Разработка моделей данных (TaskItem, перечисления)	Реализованы классы моделей данных
3	Разработка бизнес-логики	Сформирована работоспособная библиотека

Итерация	Содержание работ	Результат
	(TaskService, JsonHelper)	Core.dll
4	Разработка пользовательского интерфейса (WPF)	Реализованы главное окно и окно редактирования
5	Написание модульных тестов и отладка	Обеспечено тестовое покрытие, устранены выявленные дефекты

В процессе разработки приложения применялся следующий набор инструментов и технологий, представленный в таблице 8.

Таблица 8 – Инструментальные средства разработки

Инструмент	Версия / Описание	Назначение
Visual Studio 2022	Community Edition	Интегрированная среда разработки (IDE)
C#	.NET 6.0 / .NET Framework	Язык программирования
WPF	Windows Presentation Foundation	Технология разработки графического интерфейса
MSTest	Microsoft Test Framework	Написание и выполнение модульных тестов
Newtonsoft.Json	NuGet-пакет	Сериализация и десериализация данных в формате JSON
Git	Система контроля версий	Хранение истории изменений исходного кода

Разработка приложения осуществлялась в интегрированной среде разработки Microsoft Visual Studio 2022 Community Edition. Указанная IDE является одним из наиболее функциональных и распространённых инструментов для создания приложений на платформе .NET. Она предоставляет широкий спектр возможностей, включая интеллектуальное автодополнение кода (IntelliSense), встроенный отладчик, визуальный конструктор форм WPF, менеджер NuGet-пакетов и интеграцию с системами контроля версий.

Решение включает три проекта, организованных в единую структуру (Solution):

1. Minihanov.Praktik.Core — библиотека классов, содержащая бизнес-логику приложения. Включает каталоги Helpers (вспомогательные классы), Models (модели данных) и Services (сервисы работы с данными).

2. Minihanov.Praktik1 — основной WPF-проект, реализующий пользовательский интерфейс. Содержит каталог Views с формами AddEditTaskWindow и MainWindow.

3. Minihanov.Praktik1.Tests — проект с модульными тестами на базе фреймворка MSTest.

Архитектура приложения построена на принципе разделения ответственности (Separation of Concerns). В таблице 9 приведено описание основных классов и их взаимосвязей.

Таблица 9 – Основные компоненты архитектуры приложения

Класс / Интерфейс	Проект	Описание
TaskItem	Core	Модель задачи. Содержит поля: Id, Title, Description, Priority, DueDate, Status, IsImportant. Реализует интерфейсы INotifyPropertyChanged, ICloneable.
TaskPriority	Core	Перечисление уровней приоритета: Low, Medium, High, Critical.
TaskStatus	Core	Перечисление статусов задачи: New, InProgress, Completed.
ITaskService	Core	Интерфейс сервиса, определяющий контракт для всех операций с задачами.
TaskService	Core	Реализация интерфейса ITaskService. Обеспечивает хранение списка задач в оперативной памяти, реализует CRUD-операции и поиск.
JsonHelper	Core	Статический вспомогательный класс, предоставляющий методы SaveToFile и LoadFromFile для работы с JSON-сериализацией.
MainWindow	Praktik1	Главное окно приложения, содержащее список задач, элементы фильтрации, поисковую строку и панель статистики.

Класс / Интерфейс	Проект	Описание
AddEditTask Window	Praktik1	Диалоговое окно для добавления и редактирования задачи.
TaskManager Tests	Tests	Класс, содержащий модульные тесты, покрывающие основные операции TaskService и функциональность TaskItem.

Далее вы можете увидеть диаграмму классов

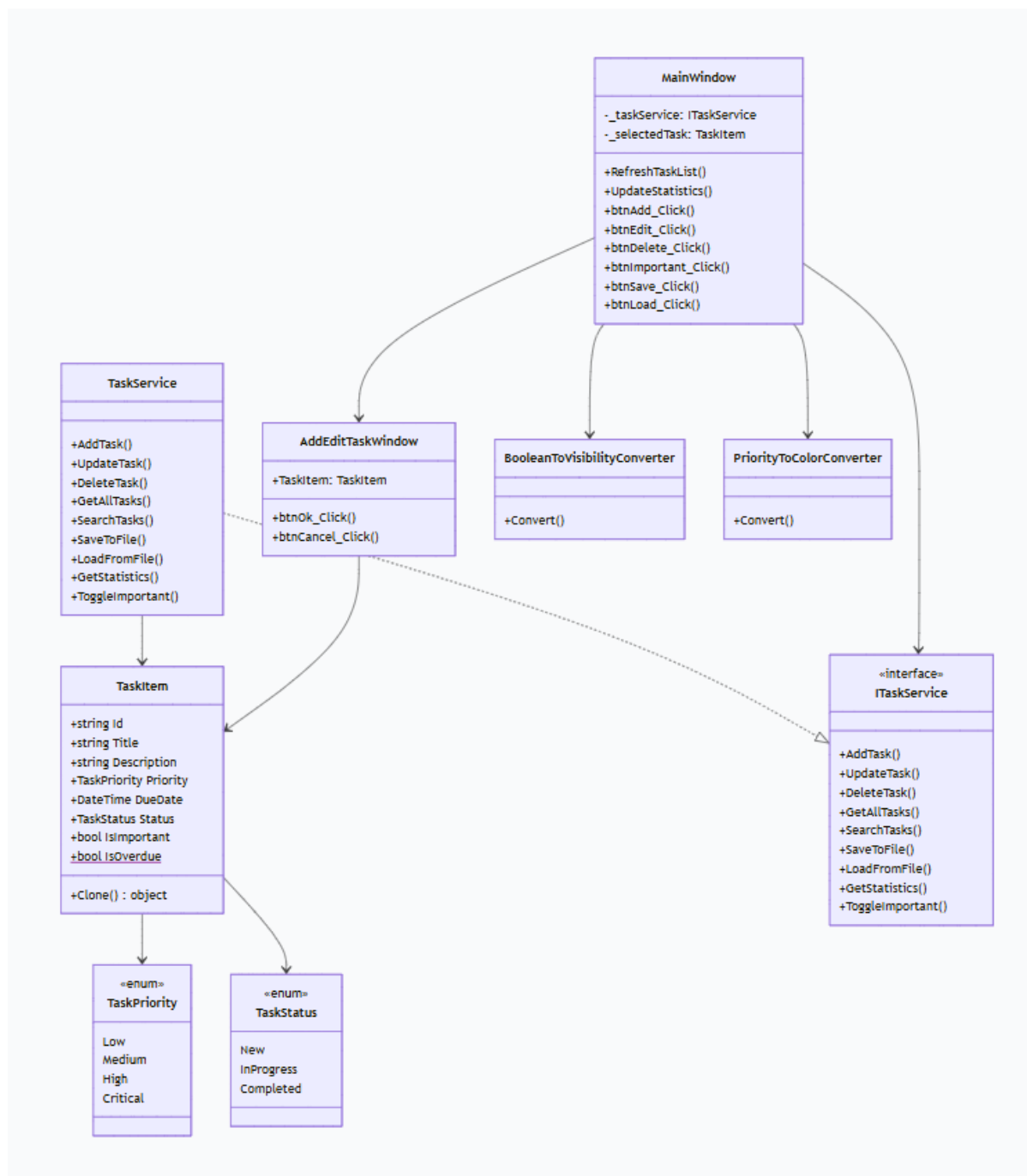


Рисунок 1 – диаграмма классов

Этапы разработки программного обеспечения

Разработка приложения «Планировщик задач» осуществлялась в несколько последовательных этапов, каждый из которых был направлен на решение конкретных задач в рамках общего процесса создания программного продукта.

1. Выбор шаблона и проектирование структуры проекта

На начальном этапе в качестве основы был выбран шаблон проекта WPF Application в среде Visual Studio. Структура решения была декомпозирована на три самостоятельных проекта с целью обеспечения чёткого разделения ответственности: библиотека классов, содержащая бизнес-логику; WPF-приложение, реализующее пользовательский интерфейс; и проект модульных тестов. Подобная архитектурная организация позволяет проводить независимое тестирование бизнес-логики без необходимости запуска графической оболочки приложения.

2. Проектирование модели данных

На данном этапе была разработана модель данных. Основным структурным элементом является класс TaskItem, инкапсулирующий все необходимые атрибуты для описания задачи. В качестве формата хранения данных выбран JSON, что обеспечивает простоту чтения файла, а также переносимость данных между различными вычислительными устройствами.

3. Разработка бизнес-логики

Был реализован интерфейс ITaskService и его конкретная реализация TaskService, предоставляющая полный набор операций для работы с задачами: добавление, обновление, удаление, поиск, фильтрация, сортировка, получение статистических данных, а также сохранение и загрузка информации. Вспомогательный класс JsonHelper инкапсулирует логику взаимодействия с файловой системой при выполнении операций сериализации и десериализации.

4. Разработка пользовательского интерфейса

В ходе данного этапа были созданы два основных окна: главное окно MainWindow, содержащее полный набор управляющих элементов, и диалоговое окно AddEditTaskWindow, предназначенное для добавления и редактирования задач. Связь пользовательского интерфейса с бизнес-логикой реализована посредством внедрения зависимости (Dependency Injection) через интерфейс ITaskService.

Далее представлен интерфейс MainWindow

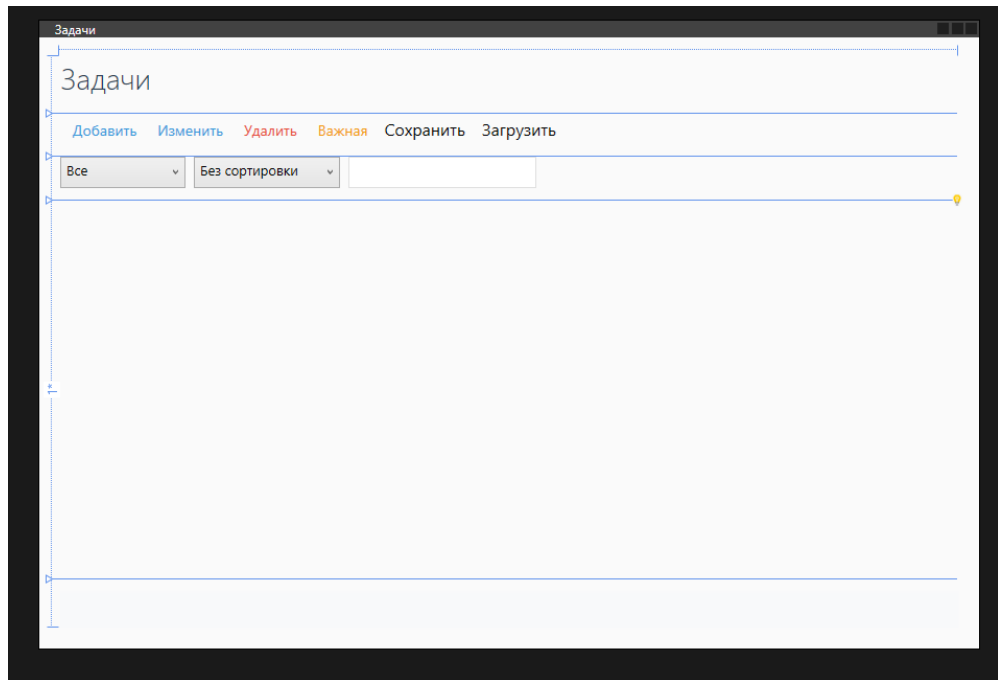


Рисунок 2 – Интерфейс MainWindow

Ниже представлен интерфейс AddEditTaskWindow

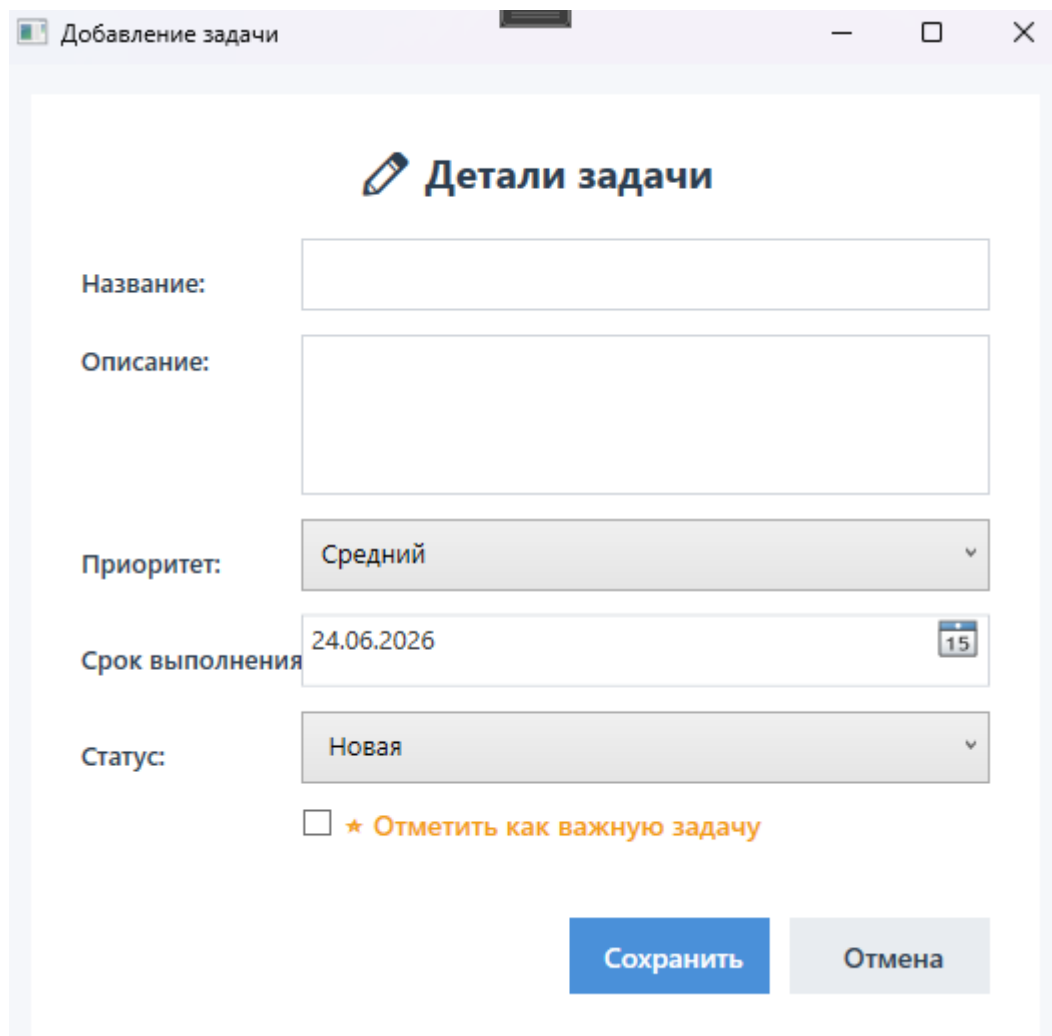


Рисунок 3 - интерфейс AddEditTaskWindow

5. Документирование системы

В процессе разработки велась техническая документация, представленная в форме настоящего отчёта. Исходный код снабжён комментариями в ключевых участках, описывающими назначение методов и логику функционирования основных компонентов системы.

6. Тестирование

По завершении этапа разработки были созданы модульные тесты, покрывающие все основные сценарии работы приложения. В процессе тестирования были выявлены и устранены ряд дефектов, в том числе связанных с некорректным определением просроченных задач.

7. Сопровождение системы

На заключительном этапе были проведены работы по исправлению выявленных ошибок: устранена ошибка `NullReferenceException`, возникавшая

при инициализации главного окна, а также скорректирована логика определения просроченности задач. Цикл сопровождения является неотъемлемой составляющей жизненного цикла любого программного продукта.

Разработка форм для ввода, просмотра и редактирования данных

В приложении реализованы две основные формы.

Главное окно (MainWindow) представляет собой основной рабочий экран приложения. В верхней части расположена панель инструментов, содержащая следующие кнопки: «Добавить», «Изменить», «Удалить», «Важная», а также кнопки сохранения и загрузки данных из файла. Ниже размещена панель фильтрации, включающая выпадающий список для фильтрации по статусу, выпадающий список для выбора критерия сортировки и поисковую строку с кнопкой очистки. Центральную область окна занимает список задач, в котором каждая задача отображается с указанием наименования, описания, статуса, приоритета, срока выполнения и отметки о важности. В нижней части окна расположена строка, отображающая статистическую информацию.

Окно добавления и редактирования задачи (AddEditTaskWindow) реализовано в виде модального диалога. Оно содержит поля для ввода всех атрибутов задачи: текстовые поля для наименования и описания, выпадающие списки для выбора приоритета и статуса, а также элемент выбора даты для указания срока выполнения. Завершение работы с диалогом осуществляется посредством нажатия кнопок «Сохранить» или «Отмена».

Этапы разработки интерфейса

Проектирование

На этапе проектирования был определён состав экранов приложения и совокупность управляющих элементов на каждом из них. Разработана логика навигации: главное окно является основным, из него открывается диалоговое окно редактирования. Определена компоновка элементов интерфейса: панель

инструментов размещена в верхней части, фильтры — под ней, список задач — в центральной области, статистика — в нижней части окна.

Прототипирование

На этапе прототипирования был создан первоначальный макет интерфейса непосредственно в XAML-дизайнере среды Visual Studio. Созданный прототип позволил оценить эргономичность расположения управляющих элементов и внести корректировки в компоновку до начала реализации программного кода логики.

Стилизация

На этапе стилизации интерфейсу придано единообразное визуальное оформление. Для элементов списка задач были настроены шаблоны `DataTemplate`, обеспечивающие визуальное выделение просроченных задач и отображение значка-звёздочки для задач, отмеченных как важные. Кнопки получили согласованный стиль оформления. Цветовая гамма приложения выдержана в сине-белых тонах, что обеспечивает профессиональный и эстетически приятный внешний вид программного продукта.

РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ

Отладка программного обеспечения представляет собой процесс обнаружения, локализации и устранения дефектов (ошибок) в программном коде. В ходе разработки приложения «Планировщик задач» применялись следующие методы и инструменты отладки.

Инструменты отладки

Основным инструментом отладки служил встроенный отладчик интегрированной среды разработки Microsoft Visual Studio 2022, предоставляющий полный набор возможностей для пошагового выполнения кода и анализа состояния программы в процессе её исполнения.

В таблице 10 представлены использованные инструменты отладки с описанием их применения.

Таблица 10 – Инструменты отладки и их назначение

Инструмент	Описание применения
Точки останова (Breakpoints)	Устанавливались в ключевых участках кода для приостановки выполнения и анализа значений переменных
Пошаговое выполнение (Step Over / Step Into)	Применялось для последовательного прохождения по строкам кода и отслеживания логики выполнения
Окно переменных (Locals / Watch)	Обеспечивало наблюдение в реальном времени за значениями переменных и полей объектов
Стек вызовов (Call Stack)	Использовался для анализа цепочки вызовов методов при возникновении исключительных ситуаций
Окно вывода (Output)	Применялось для просмотра сообщений об ошибках и диагностических сообщений
Exception Helper	Автоматически отображал информацию об исключении в момент его возникновения

В процессе отладки были обнаружены и исправлены следующие дефекты, представленные в таблице 11.

Таблица 11 – Выявленные и устранённые ошибки

Ошибка	Причина	Способ устранения
NullReferenceException в методе RefreshTaskList()	Метод вызывался из конструктора до завершения InitializeComponent(), когда элементы пользовательского интерфейса ещё не были инициализированы	Добавлена проверка условия if (_taskService == null lstTasks == null) return; в начале метода. Применён оператор ?. для безопасного обращения к элементам.
Неверное определение просроченных задач	Свойство IsOverdue сравнивало DueDate с DateTime.Now с учётом времени, вследствие чего задача со сроком, приходящимся на текущую дату, считалась просроченной в течение дня	Логика сравнения изменена на DueDate.Date < DateTime.Today, что позволяет учитывать только дату без временной составляющей
Исключение при разборе статуса задачи	Метод Enum.Parse генерировал исключение при передаче неожиданного строкового значения	Метод заменён на Enum.TryParse<TaskStatus> с обработкой случая неудачного разбора

Рассмотрим процесс отладки основной ошибки более подробно. При первом запуске приложения в отладчике Visual Studio автоматически сработал Exception Helper, указав на строку в методе RefreshTaskList(). Анализ стека вызовов (Call Stack) показал, что метод был вызван из конструктора MainWindow до завершения выполнения InitializeComponent(). С помощью окна Locals было установлено, что переменные cmbStatusFilter и lstTasks имели значение null. После добавления защитных проверок ошибка была устранена, что подтвердилось повторным запуском приложения в режиме отладки.

РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Далее рассмотрим процесс тестирования.

Таблица 12 – Тестирования

Метод теста	Проверяемая функциональность	Ожидаемый результат
AddTask_ShouldAddTaskSuccessfully	Добавление задачи в сервис	Список содержит одну задачу с корректно заполненными полями
UpdateTask_ShouldUpdateTaskSuccessfully	Обновление всех полей существующей задачи	Поля задачи изменены в соответствии с переданными новыми значениями
DeleteTask_ShouldDeleteTaskSuccessfully	Удаление задачи по идентификатору	Список задач пуст после выполнения операции удаления
GetTaskById_ShouldReturnTaskSuccessfully	Получение задачи по существующему идентификатору	Возвращается задача с идентификатором, совпадающим с запрошенным
GetTaskById_ShouldReturnNull_WhenTaskNotFound	Получение задачи по несуществующему идентификатору	Метод возвращает значение null
GetAllTasks_ShouldReturnAllTasks	Получение полного списка задач	Возвращается список, содержащий три задачи
GetTasksByStatus_ShouldReturnCorrectTasks	Фильтрация задач по статусу New	Возвращаются только задачи со статусом New
SearchTasks_ShouldReturnResultsByTitle	Поиск задач по ключевому слову в наименовании	Возвращается задача, в наименовании которой присутствует искомая подстрока
GetTasksSortedByPriority_ShouldReturnSortedTasks	Сортировка задач по уровню приоритета	Задачи отсортированы в порядке убывания приоритета (от High к Low)
LoadFromFile_ShouldLoadTasksSuccessfully	Сохранение и последующая загрузка задач из JSON-файла	После загрузки из файла список содержит те же задачи, что и до сохранения
GetStatistics_ShouldReturnCorrectCounts	Подсчёт статистических показателей по задачам	Счётчики New, InProgress, Completed, Important соответствуют добавленным задачам
ToggleImportant_ShouldToggleFlag	Переключение флага важности задачи	Значение флага последовательно изменяется с false на true и обратно
IsOverdue_ShouldReturnTrue_WhenTaskIsOver	Проверка свойства IsOverdue для просроченной задачи	Метод возвращает true для задачи со сроком

due		выполнения в прошлом
TaskItem_Constructor_ShouldInitializeDefaultValues	Проверка конструктора по умолчанию класса TaskItem	Идентификатор не равен null, статус имеет значение New, приоритет — Medium, флаг IsImportant установлен в false
TaskItem_Clone_ShouldCreateDeepCopy	Проверка метода Clone() класса TaskItem	Клон содержит те же значения полей; изменение клона не влияет на исходный объект
JsonHelper_LoadFromFile_ShouldLoadDataSuccessfully	Проверка работоспособности класса JsonHelper при сохранении и загрузке	Загруженные из файла данные полностью совпадают с сохранёнными

Далее вы увидите что все тесты которые описаны выше выполнены

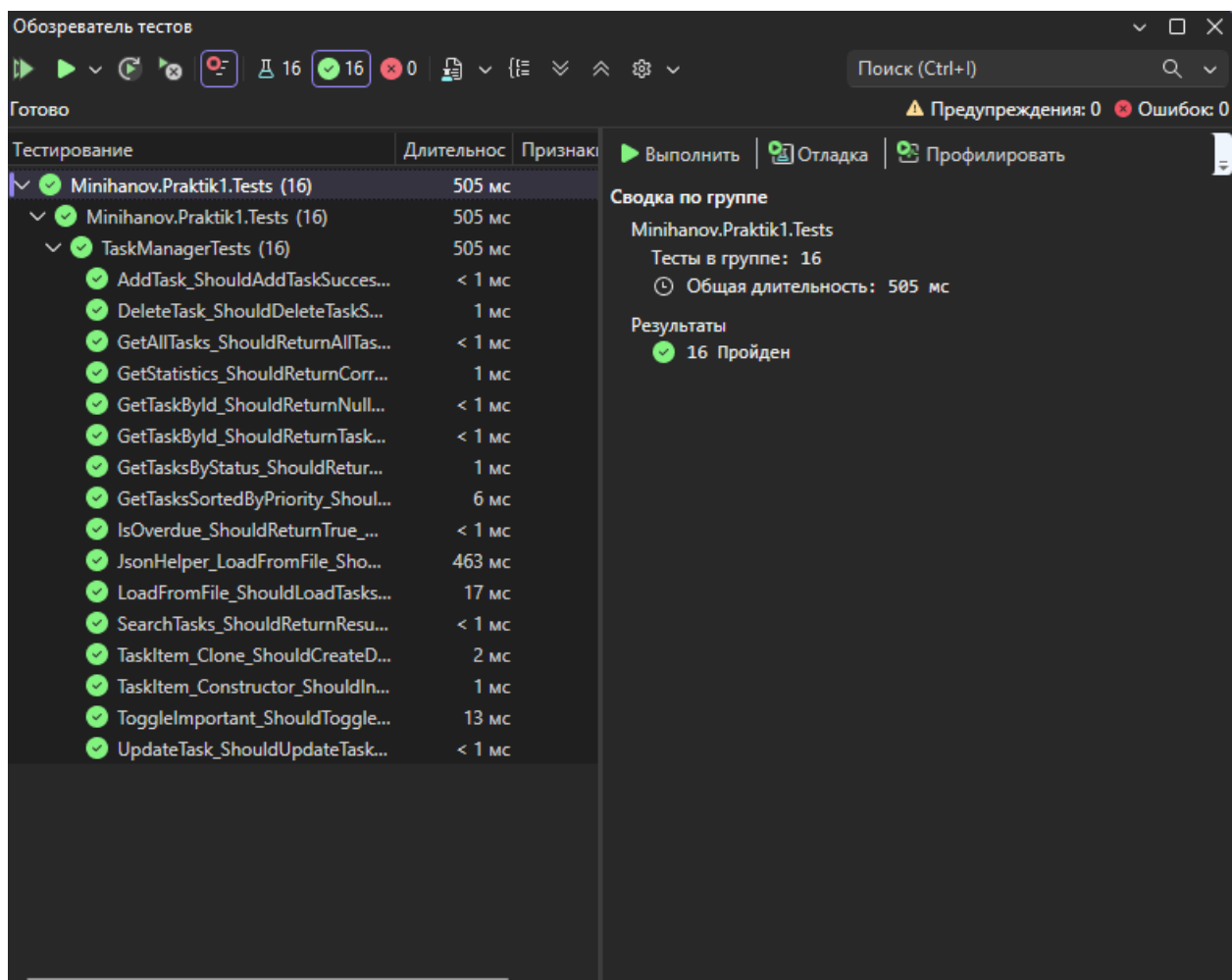


Рисунок 4- Обозреватель тестов

Ниже приведена полная структура приложения:

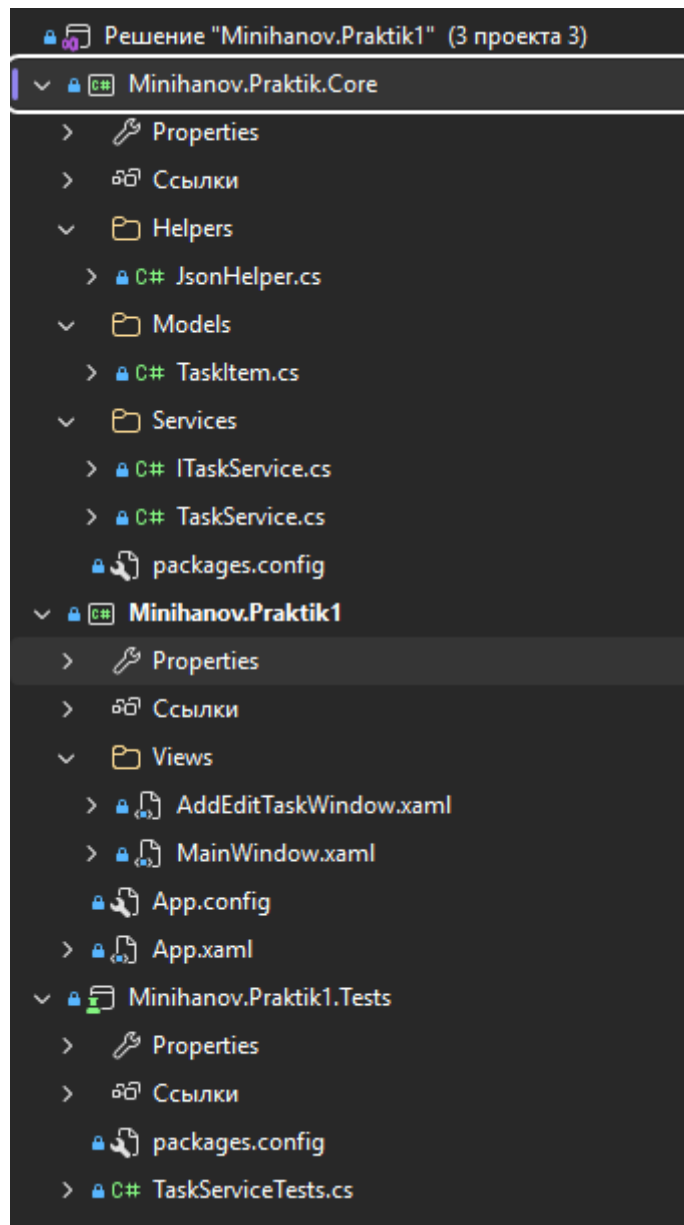


Рисунок 5 структура приложения

РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ

Инспектирование кода представляет собой процесс систематической проверки исходного кода программы на соответствие принятым стандартам и соглашениям, а также на наличие потенциальных дефектов, уязвимостей и проблем сопровождаемости. Инспектирование является неотъемлемой частью обеспечения качества программного обеспечения и позволяет выявить проблемы, которые не обнаруживаются в ходе тестирования.

В рамках настоящего проекта инспектирование проводилось как ручную — посредством просмотра кода, так и с использованием автоматизированных инструментов, встроенных в среду разработки Visual Studio.

В таблице 13 представлены виды инспектирования, применявшиеся в ходе разработки, с указанием их описания и конкретного применения в проекте.

Таблица 13 – Виды инспектирования кода

Вид инспекции	Описание	Применение в проекте
Формальная инспекция (Code Review)	Систематический просмотр кода по заранее определённым критериям. Проверяется структура, читаемость, соответствие стандартам.	Проводилась самостоятельно по завершении каждого этапа разработки
Статический анализ	Автоматический анализ кода без его выполнения. Инструменты выявляют потенциальные ошибки, предупреждения и отклонения от стандартов.	Использовался встроенный анализатор Visual Studio и рекомендации ReSharper
Сквозной контроль (Walkthrough)	Неформальный просмотр кода с целью общего понимания логики и выявления очевидных проблем.	Применялся при разработке ключевых компонентов (TaskService, MainWindow)

Вид инспекции	Описание	Применение в проекте
Проверка стиля кода	Проверка соответствия кода соглашениям об именовании, форматировании и структуре файлов.	Осуществлялась через настройки EditorConfig и правила анализатора

При разработке приложения применялись следующие стандарты и соглашения о написании кода на языке C#, представленные в таблице 14.

Таблица 14 – Стандарты и соглашения кодирования

Стандарт / Соглашение	Описание	Пример применения в проекте
Microsoft C# Coding Conventions	Официальные рекомендации Microsoft по стилю кода C#	Именование классов с заглавной буквы (TaskItem, TaskService), приватных полей с символом подчёркивания (_taskService)
SOLID-принципы	Принципы объектно-ориентированного проектирования	Выделение интерфейса ITaskService (принцип инверсии зависимостей), разделение библиотеки и пользовательского интерфейса
Соглашения об именовании .NET	Правила именования для классов, методов, свойств, переменных	PascalCase для методов (AddTask, GetAllTasks), camelCase для локальных переменных
XML-документация	Документирование публичного API с помощью XML-комментариев	Комментарии к интерфейсу ITaskService и его методам
Принцип единственной ответственности	Каждый класс решает только одну задачу	JsonHelper отвечает только за работу с файловой системой, TaskService — только за бизнес-логику управления задачами

В таблице 15 приведён перечень инструментов, использовавшихся для инспектирования кода, с указанием их типа и функционального назначения.

Таблица 15 – Инструменты инспектирования кода

Инструмент	Тип	Описание
Visual Studio Code Analysis	Встроенный статический анализатор	Встроен в Visual Studio. Выявляет предупреждения компилятора, потенциальные <code>NullReferenceException</code> , неиспользуемые переменные и другие проблемы в режиме реального времени.
Roslyn Analyzers	Анализатор на базе компилятора	Набор анализаторов, встроенных в .NET SDK. Проверяет код на соответствие рекомендациям Microsoft и общепринятым практикам C#.
ReSharper (JetBrains)	Расширение для Visual Studio	Мощный инструмент анализа кода, предлагающий рефакторинг, выявляющий избыточный код и нарушения стиля. Использовался в режиме рекомендаций.
EditorConfig	Файл настроек форматирования	Файл <code>.editorconfig</code> в корне проекта задаёт единые правила форматирования: отступы, переносы строк, кодировка файлов.
GitHub / Git diff	Инструмент контроля версий	Просмотр изменений в коде через diff позволял выявлять случайные изменения и контролировать качество вносимых правок.

По результатам инспектирования кода были выявлены и устранены следующие несоответствия установленным стандартам:

- отсутствие защитных проверок на null в методе `RefreshTaskList()` — устранено добавлением проверок в начале метода;
- использование небезопасного метода `Enum.Parse` вместо `Enum.TryParse` — исправлено на безопасный вариант с обобщённым параметром типа;
- дублирование вызова метода `UpdateStatistics()` в `RefreshTaskList()` и конструкторе — оптимизировано путём устранения избыточных вызовов;
- отсутствие обработки исключительных ситуаций при работе с файлами — добавлены блоки `try-catch` с выводом информативных сообщений для пользователя.

ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2024 г. были закреплены теоретические знания и расширены профессиональные умения.

В ходе практики было разработано настольное приложение «Планировщик задач» на языке C# с использованием технологии WPF. Приложение представляет собой полнофункциональный инструмент для управления личными и рабочими задачами, включающий возможности создания, редактирования и удаления задач, их фильтрации и поиска, сортировки по приоритету и сроку, а также сохранения данных в файл формата JSON.

По итогам практики получены следующие результаты:

Разработана архитектура приложения, состоящего из трёх проектов: библиотека бизнес-логики `Minihanov.Praktik.Core`, WPF-приложение `Minihanov.Praktik1` и проект тестов `Minihanov.Praktik1.Tests`

Спроектированы и реализованы модели данных: класс `TaskItem` с полной поддержкой `INotifyPropertyChanged` и `ICloneable`, перечисления `TaskPriority` и `TaskStatus`

Реализован сервис `TaskService`, содержащий полный набор операций CRUD, поиск, фильтрацию, сортировку и статистику

Разработан удобный графический интерфейс с главным окном и диалогом редактирования задач

Написан набор из 16 unit-тестов, покрывающих все ключевые компоненты приложения, все тесты успешно проходят

Проведена отладка приложения с использованием встроенного отладчика Visual Studio, выявлены и устранены три ошибки

Выполнено инспектирование кода на соответствие стандартам Microsoft C# Coding Conventions и принципам SOLID По окончании практики был получен практический опыт разработки требования к программным

модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

Учебная и научная литература

1. Троелсен Э., Джепикс Ф. Язык программирования C# 9 и платформа .NET 5: основные концепции и практики применения. — М.: Вильямс, 2022. — 1328 с.
2. Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. — 4-е изд. — СПб.: Питер, 2020. — 896 с.
3. Скит Дж. C# для профессионалов: тонкости программирования. — 3-е изд. — М.: Вильямс, 2019. — 608 с.
4. Прайс М. C# 10 и .NET 6. Современные кросс-платформенные разработки. — СПб.: Питер, 2023. — 832 с.
5. Албахари Д., Албахари Б. C# 10. Справочник. Полное описание языка. — М.: O'Reilly, 2022. — 1120 с.
6. Мак-Дональд М. WPF: Windows Presentation Foundation в .NET 4.5 с примерами на C# 5.0. — М.: Вильямс, 2020. — 1024 с.
7. Натан А. WPF: Windows Presentation Foundation. — СПб.: Питер, 2019. — 928 с.
8. Андерсон Ч. Основы Windows Presentation Foundation. — М.: ДМК Пресс, 2018. — 360 с.
9. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Паттерны объектно-ориентированного проектирования. — СПб.: Питер, 2020. — 448 с.
10. Мартин Р. Чистый код: создание, анализ и рефакторинг. — СПб.: Питер, 2019. — 464 с.
11. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. — СПб.: Питер, 2018. — 352 с.
12. Фаулер М. Рефакторинг. Улучшение существующего кода. — 2-е изд. — СПб.: Питер, 2020. — 448 с.

13. Ocherove R. The Art of Unit Testing with Examples in C#. — 3rd ed. — Manning Publications, 2021. — 328 p.

14. Канер С., Фолк Дж., Нгуен Е. Тестирование программного обеспечения. Фундаментальные концепции менеджмента бизнес-приложений. — М.: ДиаСофт, 2001. — 544 с.

15. Бек К. Экстремальное программирование: разработка через тестирование. — СПб.: Питер, 2003. — 224 с.

Нормативные документы

16. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — М.: Стандартинформ, 2010. — 4 с.

17. ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов. — М.: Стандартинформ, 2010. — 4 с.

18. ГОСТ 34.602-89. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. — М.: Стандартинформ, 2009. — 12 с.

19. ГОСТ Р 56920-2016. Системная и программная инженерия. Тестирование программного обеспечения. — М.: Стандартинформ, 2016. — 64 с.

Электронные ресурсы и документация

20. Newtonsoft.Json Documentation. James Newton-King. — [Электронный ресурс]. — URL: <https://www.newtonsoft.com/json/help/html/Introduction.htm> (дата обращения: 23.06.2026).

21. Microsoft Docs. C# Programming Guide. — [Электронный ресурс]. — URL: <https://docs.microsoft.com/ru-ru/dotnet/csharp/programming-guide/> (дата обращения: 23.06.2026).

22. Microsoft Docs. WPF .NET Documentation. — [Электронный ресурс]. — URL: <https://docs.microsoft.com/ru-ru/dotnet/desktop/wpf/> (дата обращения: 23.06.2026).
23. Microsoft Docs. MSTest Framework. — [Электронный ресурс]. — URL: <https://docs.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-with-mstest> (дата обращения: 23.06.2026).
24. Microsoft Docs. C# Coding Conventions. — [Электронный ресурс]. — URL: <https://docs.microsoft.com/ru-ru/dotnet/csharp/fundamentals/coding-style/coding-conventions> (дата обращения: 23.06.2026).
25. Соммервилл И. Инженерия программного обеспечения. — 10-е изд. — М.: Вильямс, 2019. — 1136 с.
26. Макконнелл С. Совершенный код. Практическое руководство по разработке программного обеспечения. — 2-е изд. — М.: Русская редакция, 2021. — 896 с.
27. Фаулер М. UML. Основы. Краткое руководство по стандартному языку объектного моделирования. — 3-е изд. — СПб.: Символ-Плюс, 2011. — 192 с.
28. Лармен К. Применение UML 2.0 и шаблонов проектирования. — 3-е изд. — М.: Вильямс, 2013. — 736 с.
29. Вигерс К., Битти Дж. Разработка требований к программному обеспечению. — 3-е изд. — М.: Русская редакция, 2014. — 736 с.
30. Липперт Э. Занимательная C#. Игры и головоломки с применением LINQ, XAML и других современных инструментов .NET. — СПб.: Питер, 2022. — 368 с.
31. GitNub репозиторий проекта «Планировщик задач»
<https://github.com/MinihanovVadimIPS-21/Minihanov.PraktikBer>

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using Minihanov.Praktik.Core.Helpers;
using Minihanov.Praktik.Core.Models;
using Minihanov.Praktik.Core.Services;
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;

namespace Minihanov.Praktik1.Tests
{
    [TestClass]
    public class TaskManagerTests : IDisposable
    {
        private readonly ITaskService _taskService;
        private readonly string _testFilePath;

        public TaskManagerTests()
        {
            _taskService = new TaskService();
            _testFilePath = Path.Combine(Path.GetTempPath(),
                $"test_tasks_{Guid.NewGuid()}.json");
        }

        public void Dispose()
        {
            if (File.Exists(_testFilePath))
                File.Delete(_testFilePath);
        }

        [TestMethod]
        public void AddTask_ShouldAddTaskSuccessfully()
        {
            var task = new TaskItem
            {
                Title = "Test Task",
                Description = "Test Description",
                Priority = TaskPriority.High,
                DueDate = DateTime.Now.AddDays(5)
            };

            _taskService.AddTask(task);
            var tasks = _taskService.GetAllTasks();
        }
    }
}
```

```

        Assert.AreEqual(1, tasks.Count());
        Assert.AreEqual("Test Task", tasks.First().Title);
        Assert.AreEqual("Test Description", tasks.First().Description);
        Assert.AreEqual(TaskPriority.High, tasks.First().Priority);
        Assert.IsNotNull(tasks.First().Id);
    }

    [TestMethod]
    public void UpdateTask_ShouldUpdateTaskSuccessfully()
    {
        var task = new TaskItem { Title = "Original Title" };
        _taskService.AddTask(task);
        var createdTask = _taskService.GetAllTasks().First();

        createdTask.Title = "Updated Title";
        createdTask.Description = "Updated Description";
        createdTask.Priority = TaskPriority.Critical;
        createdTask.Status = TaskStatus.InProgress;
        createdTask.IsImportant = true;

        _taskService.UpdateTask(createdTask);
        var updatedTask = _taskService.GetTaskById(createdTask.Id);

        Assert.AreEqual("Updated Title", updatedTask.Title);
        Assert.AreEqual("Updated Description", updatedTask.Description);
        Assert.AreEqual(TaskPriority.Critical, updatedTask.Priority);
        Assert.AreEqual(TaskStatus.InProgress, updatedTask.Status);
        Assert.IsTrue(updatedTask.IsImportant);
    }

    [TestMethod]
    public void DeleteTask_ShouldDeleteTaskSuccessfully()
    {
        var task = new TaskItem { Title = "Test Task" };
        _taskService.AddTask(task);
        var createdTask = _taskService.GetAllTasks().First();

        _taskService.DeleteTask(createdTask.Id);
        var tasks = _taskService.GetAllTasks();

        Assert.AreEqual(0, tasks.Count());
    }

    [TestMethod]
    public void GetTaskById_ShouldReturnTaskSuccessfully()
    {
        var task = new TaskItem { Title = "Test Task" };
        _taskService.AddTask(task);
        var createdTask = _taskService.GetAllTasks().First();

        var retrievedTask = _taskService.GetTaskById(createdTask.Id);

        Assert.IsNotNull(retrievedTask);
    }

```

```

        Assert.AreEqual(createdTask.Id, retrievedTask.Id);
        Assert.AreEqual(createdTask.Title, retrievedTask.Title);
    }

    [TestMethod]
    public void GetTaskById_ShouldReturnNull_WhenTaskNotFound()
    {
        var result = _taskService.GetTaskById(Guid.NewGuid().ToString());
        Assert.IsNull(result);
    }

    [TestMethod]
    public void GetAllTasks_ShouldReturnAllTasks()
    {
        _taskService.AddTask(new TaskItem { Title = "Task 1" });
        _taskService.AddTask(new TaskItem { Title = "Task 2" });
        _taskService.AddTask(new TaskItem { Title = "Task 3" });

        var tasks = _taskService.GetAllTasks();

        Assert.AreEqual(3, tasks.Count());
    }

    [TestMethod]
    public void GetTasksByStatus_ShouldReturnCorrectTasks()
    {
        _taskService.AddTask(new TaskItem { Title = "Task 1", Status = TaskStatus.New });
        _taskService.AddTask(new TaskItem { Title = "Task 2", Status = TaskStatus.Completed
    });
        _taskService.AddTask(new TaskItem { Title = "Task 3", Status = TaskStatus.New });

        var newTasks = _taskService.GetTasksByStatus(TaskStatus.New);

        Assert.AreEqual(2, newTasks.Count());
    }

    [TestMethod]
    public void SearchTasks_ShouldReturnResultsByTitle()
    {
        _taskService.AddTask(new TaskItem { Title = "Buy groceries", Description = "Milk and
bread" });
        _taskService.AddTask(new TaskItem { Title = "Write report", Description = "Monthly
report" });
        _taskService.AddTask(new TaskItem { Title = "Call plumber", Description = "Fix the
sink" });

        var results = _taskService.SearchTasks("report");

        Assert.AreEqual(1, results.Count());
        Assert.AreEqual("Write report", results.First().Title);
    }

    [TestMethod]

```

```

public void GetTasksSortedByPriority_ShouldReturnSortedTasks()
{
    _taskService.AddTask(new TaskItem { Title = "Low", Priority = TaskPriority.Low });
    _taskService.AddTask(new TaskItem { Title = "High", Priority = TaskPriority.High });
    _taskService.AddTask(new TaskItem { Title = "Medium", Priority =
TaskPriority.Medium });

    var sorted = _taskService.GetTasksSortedByPriority().ToList();

    Assert.AreEqual(TaskPriority.High, sorted[0].Priority);
    Assert.AreEqual(TaskPriority.Medium, sorted[1].Priority);
    Assert.AreEqual(TaskPriority.Low, sorted[2].Priority);
}

[TestMethod]
public void LoadFromFile_ShouldLoadTasksSuccessfully()
{
    _taskService.AddTask(new TaskItem { Title = "Task 1" });
    _taskService.AddTask(new TaskItem { Title = "Task 2" });
    _taskService.SaveToFile(_testFilePath);

    var newService = new TaskService();
    newService.LoadFromFile(_testFilePath);
    var tasks = newService.GetAllTasks();

    Assert.AreEqual(2, tasks.Count());
    Assert.IsTrue(tasks.Any(t => t.Title == "Task 1"));
    Assert.IsTrue(tasks.Any(t => t.Title == "Task 2"));
}

[TestMethod]
public void GetStatistics_ShouldReturnCorrectCounts()
{
    _taskService.AddTask(new TaskItem { Title = "Task 1", Status = TaskStatus.New });
    _taskService.AddTask(new TaskItem { Title = "Task 2", Status = TaskStatus.InProgress
});
    _taskService.AddTask(new TaskItem { Title = "Task 3", Status = TaskStatus.Completed
});
    _taskService.AddTask(new TaskItem { Title = "Task 4", Status = TaskStatus.New,
IsImportant = true });

    var stats = _taskService.GetStatistics();

    Assert.AreEqual(4, stats.Total);
    Assert.AreEqual(2, stats.New);
    Assert.AreEqual(1, stats.InProgress);
    Assert.AreEqual(1, stats.Completed);
    Assert.AreEqual(1, stats.Important);
}

[TestMethod]
public void ToggleImportant_ShouldToggleFlag()

```

```

{
    var task = new TaskItem { Title = "Test Task", IsImportant = false };
    _taskService.AddTask(task);
    var createdTask = _taskService.GetAllTasks().First();

    _taskService.ToggleImportant(createdTask.Id);
    var updatedTask = _taskService.GetTaskById(createdTask.Id);

    Assert.IsTrue(updatedTask.IsImportant);

    _taskService.ToggleImportant(createdTask.Id);
    updatedTask = _taskService.GetTaskById(createdTask.Id);

    Assert.IsFalse(updatedTask.IsImportant);
}

[TestMethod]
public void IsOverdue_ShouldReturnTrue_WhenTaskIsOverdue()
{
    var task = new TaskItem
    {
        Title = "Overdue Task",
        DueDate = DateTime.Now.AddDays(-1),
        Status = TaskStatus.New
    };

    Assert.IsTrue(task.IsOverdue);
}

[TestMethod]
public void TaskItem_Constructor_ShouldInitializeDefaultValues()
{
    var task = new TaskItem();

    Assert.IsNotNull(task.Id);
    Assert.IsTrue(task.Id.Length > 0);
    Assert.AreEqual(TaskStatus.New, task.Status);
    Assert.AreEqual(TaskPriority.Medium, task.Priority);
    Assert.IsFalse(task.IsImportant);
}

[TestMethod]
public void TaskItem_Clone_ShouldCreateDeepCopy()
{
    var original = new TaskItem
    {
        Title = "Test Title",
        Description = "Test Description",
        Priority = TaskPriority.High,
        Status = TaskStatus.InProgress,
        IsImportant = true
    };

```

```

var clone = (TaskItem)original.Clone();

Assert.AreEqual(original.Title, clone.Title);
Assert.AreEqual(original.Description, clone.Description);
Assert.AreEqual(original.Priority, clone.Priority);
Assert.AreEqual(original.Status, clone.Status);
Assert.AreEqual(original.IsImportant, clone.IsImportant);

clone.Title = "Modified";
Assert.AreNotEqual(original.Title, clone.Title);
}

[TestMethod]
public void JsonHelper_LoadFromFile_ShouldLoadDataSuccessfully()
{
    var originalData = new List<TaskItem>
    {
        new TaskItem { Title = "Task 1" },
        new TaskItem { Title = "Task 2" }
    };

    JsonHelper.SaveToFile(originalData, _testFilePath);

    var loadedData = JsonHelper.LoadFromFile<List<TaskItem>>(_testFilePath);

    Assert.IsNotNull(loadedData);
    Assert.AreEqual(2, loadedData.Count);
    Assert.AreEqual("Task 1", loadedData[0].Title);
    Assert.AreEqual("Task 2", loadedData[1].Title);
}
}
}

```

Полный листинг кода можно посмотреть по ссылке: <https://github.com/MinihanovVadimIPS-21/Minihanov.PraktikBer>